202301978 컴퓨터융합학부 김도윤

```java
import java.io.*;
import java.util.StringTokenizer;

public class Main  spicypotato0823 *
{
    public static class Node // 1  15개 사용 위치  spicypotato0823 *
    {
        private int item;  4개 사용 위치
        private Node next;  3개 사용 위치

        public Node(int item, Node node)  4개 사용 위치  spicypotato0823
        {
            this.item = item;
            this.next = node;
        }
        // 2
        public int getItem() {return item;}  4개 사용 위치  spicypotato0823
        public Node getNext() {return next;}  11개 사용 위치  spicypotato0823
        public void setItem(int item) {this.item = item;}  0개의 사용위치  spicypotato0823
        public void setNext(Node node) {this.next = node;}  3개 사용 위치  spicypotato0823
    }

```

Node 생성. Single LinkedList를 위한 노드를 만들기 위해 item과 다음 노드의 주소를 담는 next를 가지는 Node를 정의하고 (1) 이에 접근할 수 있는 method를 정의했다. (2)

```java
    public static class sList  2개 사용 위치  spicypotato0823 *
    {
        protected Node head;  13개 사용 위치

        public sList()  1개 사용 위치  spicypotato0823
        {
            head = null;
        }

        public void print()  2개 사용 위치  spicypotato0823 *
        {
            Node cursor = head; // 3

            if(head == null)// 4
            {
                System.out.println("empty");
                return;
            }
            while (cursor != null) // 5
            {
                System.out.print(cursor.getItem() + " ");
                cursor = cursor.getNext(); // 6
            }
        }
```

SList 안에 head 초기화 후 print, insert_inc, delete 함수를 선언한다. Print함수의 경우 cursor 노드로 head 값을 받은 뒤 (3) 만약 head가 null이라면 LinkedList가 비어 있음을 출력하고 (4) 아니라면 cursor값 자체가 null이 되기 전까지 cursor의 item을 출력하고 cursor를 다음 노드로 옮긴다. (6)

```java
public void insert_inc(int newItem)  1개 사용 위치  spicypotato0823 *
{
    if(head == null) // 7
    {
        head = new Node(newItem, node: null); // 새 노드를 head로 지정.
        return;
    }
    else
    {
        Node cursor = head; // 8

        if(cursor.getItem() > newItem)// 9
        {
            Node newNode = new Node(newItem, cursor);
            head = newNode;
            return;
        }
```

```java
        while (cursor.getNext() != null) // 10
        {
            if(cursor.getNext().getItem() > newItem) // 11
            {
                Node newNode = new Node(newItem, cursor.getNext());
                cursor.setNext(newNode);
                return;
            }
            cursor = cursor.getNext();
        }
        // 12
            Node newNode = new Node(newItem, node: null);
            cursor.setNext(newNode);
            return;
    }

}
```

Insert_inc 함수는

Head가 비어 있을 경우 (7), head의 item보다 작은 값이 들어올 경우 (9), 중간 값이 들어올 경우 (11), 가장 큰 값이 들어올 경우 (12)로 케이스를 구분해 각 상황에 맞도록 head 자체를 변경하거나 cursor를 이동하며 자리를 찾아 삽입할 수 있도록 했다.

```java
public void delete(int dItem)  1개 사용 위치  spicypotato0823 *
{
    while (head != null)
    {
        if(head.item == dItem)// 13
        {
            head = head.getNext();
        }
        else
        {
            break;
        }
    }
    if(head == null)
    {
        return;
    }

    Node cursor = head;

    while (cursor.getNext() != null) // 14
    {
        if(cursor.getNext().getItem() == dItem)
        {
            cursor.setNext(cursor.getNext().getNext());
        }
        else
        {
            cursor = cursor.getNext();
        }
    }
```

Delete 함수는

Head의 item 값이 삭제 대상인 값(dItem)인 경우와 아닌 경우로 케이스를 나누어 처리했다. Head의 item 값이 삭제 대상인 경우는 head의 item값과 dItem과 비교 후 같다면 head를 다음 노드로 변경해 삭제를 진행했다. 이 과정을 head가 null이 아닐 때 까지 반복해 만약 모든 노드가 삭제 대상일 경우에도 모든 노드를 삭제할 수 있도록 했다.

Head 다음의 노드들 중 삭제 대상이 있는 경우 cursor를 이동 시키며 대상 값을 찾았다. 이 때 cursor.getNext()의 item값과 비교를 진행해 만약 같을 경우 cursor의 next값을 삭제해야할 노드 다음 노드의 값으로 설정해 노드를 삭제할 수 있도록 설정했으며 바로 cursor 값을 다음 노드로 이동하지 않고 변경된 cursor의 다음 노드 item값도 비교를 진행해 비교하지 않은 노드가 없도록 설정했다.

```java
public static void main(String[] args) throws IOException  spicypotato0823 *
{
    try
    {
        File file = new File( pathname: "test1.txt");
        FileReader fr = new FileReader(file);
        BufferedReader br = new BufferedReader(fr);
        String line = br.readLine();

        while(line != null)
        {
            sList list = new sList();

            StringTokenizer st = new StringTokenizer(line);

            while (st.hasMoreTokens())
            {
                list.insert_inc(Integer.parseInt(st.nextToken()));
            }

            list.print();
            System.out.println();

            line = br.readLine();
            st = new StringTokenizer(line);

            while (st.hasMoreTokens())
            {
                list.delete(Integer.parseInt(st.nextToken()));
            }
            list.print();
            System.out.println();
            line = br.readLine();
        }
        br.close();
```

sList를 생성하고 첫번째 줄에 있는 데이터를 insert_inc 함수를 활용해 연결 리스트에 삽입하고 다음 줄에 있는 데이터를 delete 함수를 활용해 삭제를 진행했다.

```
/Library/Java/JavaVirt
1 2 3 4 5
1 2 4 5
0 1 1 2 4 4 5 7 8
0 2 5
5 5 5 5 5
empty
```

: 출력물